A simpler elimination rule for intuitionistic primitive recursive arithmetic

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

The current native scalar presentation of iPRA uses equality transport and conditional-universal elimination as two overlapping possibilities in its proof checker. We prove that primitive equality transport need never start at the literal term 0. The missing case is recovered internally by equation induction, a fixed scalar singleton test, and induction on formula syntax. This permits the source term to select a unique elimination major. The resulting calculus has exactly the same native consequence relation, including arbitrary retained assumptions, and therefore inherits the whole-domain bi-interpretation with full first-order intuitionistic PRA.

Three complete Python checkers improve the current attained minima from 261 to 259 significant tokens, from 289 to 283 AST nodes, and from 529 to 526 normalized bytes. These are separate profiles, not the size coordinates of one program. The input and verdict contracts are unchanged. The archive includes the full mathematical derivation, the inherited arithmetic bridge, an executable proof transformation, concrete certificates, comparison sources, and reproducible measurements. No absolute minimality or proof-assistant verification of the new theorem is claimed.

Contents

1 Result and comparison boundary **3**

2 The complete proposed calculus **4**

3 The mechanism of the reduction **5**

4 Removing literal zero from the source of transport **6**

5 Whole-domain bi-interpretability with iPRA **11**

6 Complete checkers and their trust boundary **12**

7 Executable evidence and reproducibility **14**

8 Scope, provenance, and remaining limits **15**

Inherited foundations reproduced with attribution **16**

A Complementary introductions with one local assumption **16**

B Arithmetic strength and both ordinary interpretation composites **21**

C Internal scalar predecessor bootstrap **23**

D From literal-equation induction to full intuitionistic PRA **27**

E Pairing, primitive recursion, and internal numerical composites **32**

1 Result and comparison boundary

The baseline is the public `ConciseIPRA` directory at the following pinned snapshot [1]:

Commit: `b17dd5487f34adea473a0204c5320fb3eeb8910e`
Research tree: `1770db22fe06b1e5c3089b557d9165625a133574`

It contains twenty-two research packages and a Lean development of the native calculus and its interpretation into iPRA. The mathematical language and arithmetic basis used here are inherited from that development.

The new theorem. The primitive rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

may be restricted to $s \not\equiv 0$, where this is a *syntactic* test. For example, $I(2, 1; u.0)$ is an allowed source although it denotes zero. The restriction does not ask the checker to decide an arithmetical inequality. Every omitted rule instance is derivable.

The implementation consequence. For canonical certificates the source is falsy exactly when it is the literal numerical zero. The equality and conditional-universal branches can therefore use the single comparison

```
m == ([s, u] if s else [t] * 3)
```

instead of comparing against both possible major patterns. In particular, the correctness of this change does not rely on recoding zero as a truthy scalar or on assuming a missing zero-source transport rule.

Complete module	Tokens	AST nodes	Bytes	Gzip-9
Byte predecessor	266	310	529	333
New byte profile	266	305	526	330
AST predecessor	263	289	613	349
New AST profile	263	283	607	348
Token predecessor	261	292	576	345
New token profile	259	289	572	339

Table 1: Matched comparisons on CPython 3.12.14. Bytes are UTF-8 source after `ast.unparse` normalization and a final newline. Gzip-9 uses `mtime=0` and is informational.

The frozen comparison set contains 56 primary checker modules from the current research packages. Its attained coordinate minima agree with the repository's documented frontier. An attained minimum in this finite set is not a lower bound on all possible proof checkers. All counts include the complete parser/driver, substitution machinery, arithmetic instructions, inference checking, and `ACCEPT` output. The Python interpreter and its standard JSON library are shared runtime dependencies, as in the baseline. No proof database, translator, or producer is imported by a selected checker.

Each selected program also improves its matched predecessor without making any of the three main coordinates worse. The byte and AST profiles compose the new matching rule with a small ordering/fusion change. The byte profile also reuses the checker function as an invalid substitution sentinel. These two implementation changes are distinguished from the admissibility theorem in Section 6.

What is preserved. The numerical sort, actual equality, scalar iterator, native binders, induction restriction, and all old theorems are preserved. Both interpretation domains remain the whole numerical domain, and both numerical composites have the internally provable identity graph $x = y$. The contribution is a smaller primitive elimination interface, rather than a replacement of the scalar arithmetic foundation.

2 The complete proposed calculus

2.1 Terms, formulas, binding, and falsity

There is one numerical sort. Its terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in the update h; the counter n and initializer a are outside that binder. All other variables in h are fixed numerical parameters. The iterator has no access to a varying counter in its update. Every term is finite syntax. In an ordinary first-order realization, each fixed earlier-stage update description gives a function symbol with its displayed numerical parameters; there is no universal evaluator.

Native formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The second constructor binds y, and the third binds distinct variables x, y. Substitution is capture-free, after renaming bound variables. Contexts are finite lists of native formulas, with reusable assumptions. The designated contradiction is $\bot_* := (0 = S0)$. There is no separate primitive falsity-elimination rule. It is derived after full transport is recovered, by the inherited reconstruction of intuitionistic logic.

The intended ordinary meanings are

$$Q_y B(y) := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

These meanings specify the interpretation, rather than adding ordinary logical rules to the native checker.

2.2 All primitive axioms and rules

In every retained context the two computation schemas are

$$I(0, a; u.h) = a,$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

There are no additional primitive successor, equality, or arithmetic axioms.

Hypothesis. If $A \in \Gamma$, infer $\Gamma \vdash A$.

Elimination with a source. For a native template $F(v)$ define the required major by finite syntax:

$$M(s, t; F) := \begin{cases} Q_v F(v), & s \equiv 0, \\ s = t, & s \not\equiv 0. \end{cases}$$

The single elimination pattern is

$$\frac{\Gamma \vdash M(s,t;F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{SE}$$

Thus SE is exactly QE when s is literal zero, and restricted transport otherwise. In the proof below it is convenient to refer to these two specializations as QE and Tr^{-}.

Paired elimination.

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

Here b is fresh for Γ, A and the fixed parameters of B.

Paired introduction by cases.

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}. \tag{DI-case}$$

The fresh parameter z is absent from Γ, t and the fixed parameters of B. The displayed base assumption is discharged.

Paired introduction by equation progress.

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}, \qquad E \text{ a literal equation.} \tag{DI-progress}$$

The same freshness conditions hold, and the displayed equation assumption is discharged. An equation under a binder is not a literal equation.

This is the complete inventory. Exchange, weakening, contraction, cut, renaming, reflexivity, ordinary logical connectives, numerical cases, equation induction, and eventually unrestricted numerical substitution are derived. In the initial proof of the reduction, arbitrary numerical substitution is deliberately not assumed: it could turn a permitted variable source into the forbidden literal zero.

3 The mechanism of the reduction

The difficulty is not the syntactic test. From $0 = t$, a direct attempt to reverse equality normally transports the reflexivity of zero, which is exactly the removed case. A general existential naming argument is also unsafe as a starting point: the standard formula-selector proofs already use full equality transport. The following arithmetic construction avoids that dependency.

Use $1 = S0$, $2 = S1$, and write

$$C(n,a,b) = I(n,a;u.b), \quad Z(n) = C(n,1,0),$$

$$N(n) = I(n,0;u.Z(u)), \quad R(n) = I(n,0;u.C(u,1,2)), \quad L(n) = N(R(n)).$$

These are fixed scalar terms. Before any predecessor construction, the restricted calculus proves

$$L(1) = 1, \qquad L(SSn) = 0, \qquad L(Sn) = Z(n), \qquad n = C(Z(n),n,0).$$

The case where two nonempty expressions both compute to zero needs a separate joining lemma. It is proved using the common nonempty iterator $I(0,0;u.0)$; it does not silently use transitivity through zero.

For a fixed $t \not\equiv 0$, retain $0 = t$ and perform equation induction on

$$E(n): \quad n = I(n, t; u.Su).$$

Its base uses the allowed source t, and its step uses a fresh variable as source. At $n = 1$ this yields $1 = St$. Applying the singleton test and then the unconditional filter identity gives the proof chain

$$\boxed{0 = t \quad \Longrightarrow \quad 1 = St \quad \Longrightarrow \quad Z(t) = 1 \quad \Longrightarrow \quad t = 0.}$$

Every displayed intermediate use of transport has a variable, successor, or iterator as source. The assumption $0 = t$ stays fixed during induction. The numerical case proofs are unconditional, so no retained assumption is silently changed during a case split.

This establishes unrestricted symmetry. Numerical congruence and equality transitivity follow, with the zero-middle case handled by the joining lemma. Transport for an equation template then follows from that equality algebra. External induction on the formula template recovers transport through Q and D, using only case introduction and the corresponding eliminations. The next section supplies the full derivation and its computation-certificate compatibility proof.

4 Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem 4.1 (Literal-zero source elimination)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ *is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.*

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

4.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma 4.2 (Cases and equation induction need no transport)**.** *In $\mathcal{C}^-$ the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}\ \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathsf{EI}.$$

In NC, A is any native formula; in EI, E is one literal equation. The parameter z is fresh for Γ, t, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_yA(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

4.2 The equality operations available immediately

Lemma 4.3 (Reflexivity and safe equality operations)**.** *Reflexivity $r = r$ is derivable for every term r. From $p = q$ with $p \not\equiv 0$, one can derive $q = p$ and $H(p) = H(q)$ for any numerical template H. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma 4.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

4.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v,a,b) &:= I(v,a;u.b), & Z(v) &:= C(v,1,0),\\ K(v) &:= C(v,0,1), & H(v) &:= C(v,1,2),\\ N(n) &:= I(n,0;u.Z(u)), & R(n) &:= I(n,0;u.H(u)),\\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma 4.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma 4.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma 4.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

4.4 Turning a zero-source equality around

Lemma 4.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma 4.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives $Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

4.5 Recovering equality algebra and formula transport

Lemma 4.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma 4.3 when the source is not zero and Lemma 4.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma 4.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma 4.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_yB(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives $B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem 4.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma 4.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

4.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences

by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

4.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

5 Whole-domain bi-interpretability with iPRA

The admissibility theorem preserves every native sequent. Thus it preserves the established interpretation bridge exactly, rather than merely preserving the truth of numerical equations in the standard model. We spell out that bridge and reproduce its mathematical ingredients in the appendices.

5.1 The ordinary scalar theory

Let T_{eq} be intuitionistic first-order logic with actual equality over the scalar signature, with both computation schemas, coverage

$$\forall n\,(n = 0 \vee \exists m\, n = Sm),$$

separation $(0 = S0) \to \bot$, and contextual induction for literal equations. The separation clause is essential: positive equations alone admit a singleton structure. It is explicit in the ordinary realization, while native falsity is the equation $0 = S0$ itself.

The translations of Q, D are those in Section 2. Restricted transport is sound in ordinary equality logic. DI-case is sound by coverage, and DI-progress by equation induction under its conditional base. Conversely the new theorem recovers the original full transport rule; the native logical macros then derive all rules and axioms of T_{eq}. The ordinary and native translations have provable formula round trips, including formulas nested under either binder.

5.2 From scalar arithmetic to all primitive recursive descriptions

The inherited proof proceeds in a necessary order:

1. Construct scalar predecessor P and prove $P0 = 0$, $P(Sn) = n$ by literal-equation induction and numerical cases. No pairing or equality detector is presupposed.

2. Construct truncated subtraction and a Boolean characteristic of actual equality on the whole domain. Compile each fixed quantifier-free formula into a characteristic equation, recovering quantifier-free induction.
3. Define pairing and projections using bounded square-root iteration, then compile every fixed primitive-recursive description into a scalar term.

Write π_0, π_1 for the two derived pairing projections, distinct from the singleton-test notation above. For a target recursion with base g and update h, the last step uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(S\pi_0(u), h^*(\pi_0(u), \pi_1(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = \pi_1(V(n, \vec{p})).$$

All passive parameters stay fixed, and $\pi_0(V(n, \vec{p})) = n$ is an internal equation-induction theorem. This is compilation of finite descriptions, not postulation of a universal evaluator.

In the other direction, iPRA interprets every scalar iterator by ordinary primitive recursion with a counter-independent update. Internal uniqueness of the recursively defined operations proves the term round trips. Both domain interpretations are $x = x$, equality stays actual equality, and the composite isomorphisms are the graphs $x = y$. Formula round trips follow by structural induction. Hence full first-order intuitionistic PRA and the proposed calculus are bi-interpretable and equiconsistent.

The appendices reproduce the finite-state predecessor bootstrap, the equality-reflection proof, the primitive-recursion compiler, and the logical macros with attribution [2, 3, 4, 5, 6]. The pinned Lean files `NativeComposite.lean` and `BiInterpretation.lean` also state the existing native/T_{eq} and T_{eq}/iPRA equivalences [12]. The new admissibility theorem has not been formalized in Lean or Rocq, and the repository's Lean build was not rerun for this report.

6 Complete checkers and their trust boundary

6.1 Public contract and canonical expressions

Each selected checker is a standalone Python module. It reads a JSON array from standard input. The final element is a closed goal, and preceding elements are instruction packets consumed from right to left. An unused prefix is inert. Acceptance means normal exit with exactly `ACCEPT` followed by a newline; an exception or nonzero process status is rejection. The checks do not depend on Python assertions, so the same contract holds under `python3 -O`. Resource exhaustion is rejection; as with the baseline, the mathematical completeness statement abstracts from a fixed machine's finite time, memory, and recursion limits.

The canonical old zero encoding is retained: scalar JSON `0` denotes numerical zero, `[0]` denotes $S0$, unary formulas encode D, binary formulas encode equality, and a canonical Q resource consists of three copies of its body. All variables are numerical. Bound-coordinate slots have width two; numerical root depth is three. The common opening traversal adds $5-\mathrm{arity}$ under an array. Successor and the iterator's non-update children carry the inherited dummy zero slots. Admitted negative integers are rigid fresh parameters. This detailed codec is inherited; its full sorted interpretation and proof are supplied in `reference/bridge/inherited_arity_checker.tex` [7].

For the main and byte profiles, an instruction is $[m, s, u, t]$:

Role	Fields and effect
Equality elimination	s truthy; require $m = [s, u]$ and the target instance of template t; check m and continue from the source instance.
Conditional elimination	s falsy; require $m = [t, t, t]$; check m and continue from the zero instance of template t.
Paired elimination	A proved unary major exports its three-copy Q resource at a fresh numerical parameter.
Computation	$m = []$, u truthy; s supplies zero or a successor predecessor, u is the initializer, and t is the update.
Case introduction	$m = []$, $u = []$; t is the existential witness.
Equation progress	$m = []$, $u = 0$; t is the witness, and only a literal equation receives a genuine induction assumption.

The token profile inherits a global fresh-name supply and puts the case/progress marker in m, as its matched predecessor does. Its producer therefore uses the corresponding global protocol. This difference is documented in `code/FORMAT.txt`; the three programs are not claimed to accept identical raw byte streams.

6.2 Why source dispatch is complete

On canonical numerical syntax, a nonzero source has a truthy representation: variables are negative rigid coordinates or positive protected coordinates, and successor/iterator terms are nonempty arrays. Thus the conditional major comparison implements exactly SE. Theorem 4.1 restores every omitted source-zero instance.

Computation packets exclude a literal-zero initializer and a literal-zero successor predecessor. They are recovered with the inherited nonempty zero $c_0 = I(2, 1; u.0)$, using only transport at source c_0. Section 4.6 verifies this compatibility before appealing to the new admissibility theorem. A proof transformation first eliminates forbidden transport sources, then expands computation leaves, then emits scoped packets. None of these larger untrusted producers is loaded by the checker.

6.3 Why accepted raw aliases remain sound

The soundness argument is semantic, rather than a claim that every raw tree is canonical or that opening is idempotent. At numerical positions, arrays of arity zero, two, or at least four are zero annotations; at formula positions, arity zero and arities at least four denote the fixed truth $0 = 0$. Successfully admitted scalar formula padding also denotes truth. A ternary formula reads its last child as a Q matrix. The frozen codec proof gives capture-free semantic opening at every meaningful sorted position.

If the new major comparison succeeds with a truthy source, the proved major is an actual equality, and the replacement is ordinary sound transport. If the source is falsy, the proved major is the specified Q resource and the replacement is its zero-instance elimination. The target match must also succeed. If it does not, no goal conversion occurs. Tripling a proved unary major gives its Q resource; tripling any other nonempty array yields a truth annotation. No failed major proof is accepted as an instruction marker. A raw truthy alias may denote arithmetic zero; any resulting equality transport is sound in the full native calculus, which Theorem 4.1 identifies with the reduced one.

Fresh names and contexts retain the baseline discipline. The main profile extends tuples and the byte profile constructs new lists; neither mutates a caller's assumption context. Witnesses

are opened before allocating the next fresh parameter. For generated computation hypotheses, simultaneous zero specialization of fresh names, rather than a false assertion of uniform truth in those names, justifies the continuation. Fractional fresh aliases, when a raw successful input admits them, receive the same simultaneous-specialization treatment as in the frozen codec audit.

6.4 Two small implementation compositions

In the byte and AST profiles, the pure calculation of a possible replacement goal precedes the recursive major proof. The context and lexical scope floor do not change during that calculation. A successful major check returns `None`; consequently it can be the first operand of the already existing `or` expression for the exported hypothesis. Failure still rejects the whole certificate. This removes a separate expression statement without omitting a proof obligation.

The byte profile uses the existing function object `c` as the default invalid numerical substitution image. JSON cannot supply that callable, and attempting to use it as a numerical image raises before it can become an accepted numerical image, just as the previous complex sentinel rejected. This is a one-character source saving with no new trusted helper.

6.5 The selected AST profile, in full

```
from json import load

def c(a, g=([],), k=0):

    def i(a, t=None, d=0):
        try:
            return [i(b, t, d + 5 - len(a)) for b in a]
        except:
            return a if k <= a < d else d % 2 and i(t, d=3)
    while i(a) not in g:
        m, s, u, t = q()
        if m == ([s, u] if s else [t] * 3) and i(t, u) == a:
            a = i(t, s)
        h = c(m, g, k) or m * 3 or ([[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else
     (a := i(*a, t, 2)))
        k -= 1
        g += (i(h, (m or len(a) % 2 == u) and k),)
        a = i(a, [k])
c((q := load(open(0)).pop)())
print('ACCEPT')
```

The readable, self-contained expansion is supplied separately. The compact program's size is not obtained by excluding that expansion from a hidden dependency: it does not import it or any other project file.

7 Executable evidence and reproducibility

The packaged reproduction run passed all **212 expected-verdict checks** on CPython 3.12.14, using both normal execution and `python3 -O`. This comprises 148 expected acceptances and 64 expected rejections across three compact checkers, their readable expansion, and two frozen baseline comparators for the old-certificate regression. The suite includes:

- 120 runs of fifteen named native proof cases (fourteen positive and one forbidden compound-progress case), after removal of zero-source transport and expansion of computation leaves;

- 48 rejections of six raw controls, covering falsehood, an unallocated parameter, fractional/future-variable capture, a false scalar sentinel, and an invalid phase-guard continuation;
- 32 acceptances of four derived examples in the appropriate local or global packet dialect, including the zero-symmetry construction and transport under Q and D;
- 12 runs comparing the old contextual-induction certificate: four baseline acceptances and eight rejections by the source-restricted checkers.

All 56 frozen comparison modules passed their SHA-256 identity check. The complete verdict receipt is `evidence/reproduced/checks.json`; the individual derived-certificate sizes and source audits are in `tests/certificates/validation.json`.

The central regression is a previously accepted contextual-induction proof whose old certificate uses a literal-zero-source transport. Directly changing the matcher correctly rejects that old instruction. The new proof transformation produces an accepted certificate of the same conclusion. This distinguishes preservation of the theorem relation from preservation of old serialized proofs. Further certificates exercise the recovered transport under both Q and D.

The new certificate generator audits its expanded native proof trees and reports zero primitive transport nodes with a literal-zero source. It uses the retained source c_0 for zero-computation repair. The normalizer is untrusted: the standalone checker verifies every emitted certificate. Finite executions provide implementation evidence; the universal equivalence and bi-interpretability statements rest on the displayed mathematical proofs.

Run the following from the extracted archive directory:

```
python3 reproduce.py
python3 code/checker.py < tests/certificates/contextual_induction_local.json
python3 -O code/checker_bytes.py < tests/certificates/transport_under_Q_local.json
python3 tools/measure.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The reproduction script uses only the Python standard library. It reruns the certificate expectations, measures the frozen comparison programs and selected kernels, checks their source identities, and writes its receipts under `evidence/reproduced/`. LaTeX compilation is a separate optional step because the compiled PDF is already included.

8 Scope, provenance, and remaining limits

This is a strict but small improvement under the repository's checker-size metric. Longer untrusted derivations replace primitive rule cases. No optimality of certificate length or checking time is claimed, and different programs attain the three new minima.

Scalar arithmetic, the Q/D macros, the arity codec, certified-zero repair, and the iPRA bridge are inherited. The repository's truthy-zero encoding and constructed-major protocol [10] are distinct earlier techniques; the new theorem works with the old canonical zero encoding.

The repository-local comparison covered the package inventory and 107 main or theory LaTeX modules. The earlier zero-source-only obstruction concerns the opposite restriction [11]. No literature-wide priority, absolute simplest theory, or universal source-size lower bound is claimed.

The new theorem and codec composition have written proofs, separate assistant reviews, and finite executable evidence. Independent peer review and proof-assistant formalization remain outstanding. The cited upstream Lean bridge was not rebuilt, and this report supplies no internal consistency proof for iPRA.

Inherited foundations reproduced with attribution

The following five modules are reproduced from the pinned `certified_zero/reference` directory under the repository's MIT-0 license. Their mathematical contributions are inherited. They provide the logical and arithmetic details of the bi-interpretation used above. Their internal historical comparisons concern earlier members of the same series, not new claims of this report. Their symbols are local to the respective modules. The unmodified reference files and their provenance are included in the archive.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$

the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [4] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n,a;h) = I(n, I(m,a;h);h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy,a;h) = I(y,h(a);h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y,a;h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y,3a;H) = 3I(y,a;H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y,a;H)) = 3H(I(y,a;H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y,1;H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x,1;u.2u), \qquad L(x,y) = N\big(I(y,E(x);H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy,2E(x);H)\big) \\ &= N\big(I(y,H(2E(x));H)\big) \\ &= N\big(I(y,3E(x);H)\big) \\ &= N\big(3I(y,E(x);H)\big) \\ &= N\big(I(y,E(x);H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m),u,Su), \qquad W(t,m) = I(t,0;u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, repository snapshot `b17dd5487f34adea473a0204c5320fb3eeb8910e`.
Pinned repository snapshot on GitHub.

[2] ProveIt, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, pinned snapshot above.

[3] ProveIt, *Arithmetic strength and both ordinary interpretation composites*, `Research/certified_zero/reference/inherited_ordinary_bridge.tex`, same snapshot.

[4] ProveIt, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, same snapshot.

[5] ProveIt, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, same snapshot.

[6] ProveIt, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, same snapshot.

[7] ProveIt, *A uniform arity encoding and its checking theorem*, `Research/certified_zero/reference/inherited_arity_checker.tex`, same snapshot; supplied as a reference file.

[8] ProveIt, `Research/constructed_computation/`, same snapshot.

[9] ProveIt, `Research/prevalidated_elimination/`, same snapshot.

[10] ProveIt, *A uniform zero representation and constructed majors*, `Research/uniform_zero/new_theory.tex`, same snapshot.

[11] ProveIt, *Why zero-source transport alone is insufficient*, `Research/monus_and_neutral_major/theory/obstruction.tex`, same snapshot.

[12] ProveIt, `Lean/ConciseIPRA/NativeComposite.lean` and `Lean/ConciseIPRA/BiInterpretation.lean`, same snapshot.